

PRÁCTICA RFID

Comunicación lectura y escritura en credencial RFID con ESP32-C6 y
modulo RC522 usando MicroPython

Sistemas Embebidos

MicroPython · RFID

1. Objetivo

El objetivo de esta práctica es que el alumno conozca y aplique el funcionamiento básico de los sistemas de identificación por radiofrecuencia (RFID), implementando su configuración mediante el módulo RC522 en conjunto con el microcontrolador ESP32-C6, con el fin de desarrollar un sistema de control de acceso basado en la lectura de tarjetas RFID (RFID Card Tag).

De manera complementaria, se integran los siguientes conceptos:

- Configuración e inicialización del módulo RFID RC522 mediante comunicación SPI con el ESP32-C6.
- Lectura e interpretación del identificador único (UID) de tarjetas RFID.
- Validación de credenciales para control de acceso a un sistema.
- Manejo de periféricos digitales del microcontrolador (GPIO).
- Programación en MicroPython para la interacción con hardware externo.
- Comunicación serial para monitoreo de eventos mediante la terminal de Thonny IDE.
- Implementación de lógica de control para permitir o denegar acceso según la tarjeta detectada.

1.1 Objetivos Específicos

- Conocer el hardware del módulo RFID RC522 y sus características principales.
- Comprender el funcionamiento del módulo RC522 y su protocolo de comunicación con el microcontrolador.
- Implementar la lectura y escritura de datos en tarjetas y credenciales RFID.
- Analizar métodos de validación de comunicación entre el módulo RFID y el ESP32-C6.
- Identificar y aplicar técnicas de detección y reconocimiento de credenciales RFID.
- Explorar los diferentes modos de configuración del sistema de lectura/escritura RFID.
- Conocer y utilizar las funciones principales de la librería MFRC522 para el manejo del módulo.

1.2 Alcance de la Práctica

El alcance de esta práctica consiste en desarrollar un sistema de identificación por radiofrecuencia (RFID) utilizando el módulo RC522 en conjunto con el microcontrolador ESP32-C6 programado en MicroPython. A través de esta implementación, se busca comprender de manera práctica cómo se relaciona el hardware con el software para lograr la comunicación entre ambos dispositivos mediante el protocolo SPI.

2. Hardware Utilizado

2.1 Componentes

Componente	Cantidad	Descripción
ESP32-C6	1	Microcontroladores principales con soporte WiFi y ESP-NOW
Cable USB-C	1	Programación y alimentación
PC con Thonny IDE	1	Entorno de desarrollo y consola serial
Modulo RC522	1	Lector RFID basado en comunicación SPI
RFID Tag	1	Tarjeta RFID para pruebas de lectura/escritura
RFID Key Fob	1	Llavero RFID para pruebas de autenticación
Cable de protoboard	1	Conexiones eléctricas entre módulos
Librería MRFRC522	1	Librería en MicroPython para control del módulo RFID
Proto Board	1	Conexiones

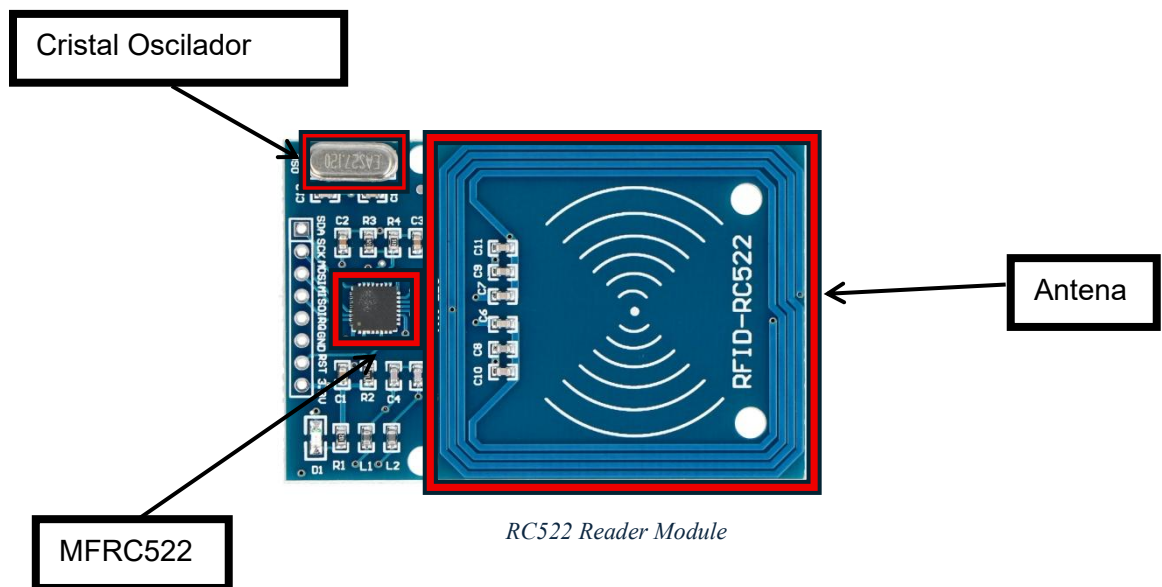
Las partes principales que incluye un sistema de lectura RFID son: RFID Reader Module, RFID Card Tag, Key Fob Tag.



2.2 RC522 Reader Module

El módulo RC522 Module cuenta con diversos componentes electrónicos que permiten la transmisión y recepción de información hacia dispositivos RFID, como las RFID Card Tag y RFID Key Fob. Estos componentes trabajan en conjunto para establecer la comunicación mediante radiofrecuencia y garantizar el correcto funcionamiento del sistema. Entre los principales componentes se encuentran:

- **Antena:**
Es el elemento encargado de transmitir y recibir señales de radiofrecuencia hacia las tarjetas o llaveros RFID. Además, genera el campo electromagnético necesario para suministrar energía a los dispositivos RFID pasivos y permitir la comunicación inalámbrica.
- **Cristal Oscilador:**
Permite establecer la frecuencia de operación del sistema y sincronizar el funcionamiento interno del módulo. Este componente proporciona la señal de reloj necesaria para el procesamiento y transmisión correcta de la información.
- **Microcontrolador MFRC522:**
Es el circuito integrado principal del módulo y se encarga de controlar todo el proceso de comunicación RFID. Administra el envío y recepción de datos, controla la modulación y demodulación de señales de radiofrecuencia y permite la comunicación con tarjetas compatibles con los estándares [ISO/IEC 14443 A](#) y MIFARE sin requerir circuitería activa adicional.



Este módulo funciona mediante una antena integrada que le permite “comunicarse” con las tarjetas RFID sin contacto físico. Cuando está encendido, el lector está constantemente buscando señales en su entorno.

Si una tarjeta o llavero RFID se acerca lo suficiente —aproximadamente hasta unos 50 mm— el módulo la detecta automáticamente. A partir de ese momento, reconoce que hay una credencial dentro de su rango y puede leer su información, identificando así su presencia de forma inmediata y sin necesidad de intervención manual.

Este microcontrolador integrado dentro del módulo de detección RFID es el encargado de gestionar todo el proceso de control de energía, así como la lectura y escritura de datos en las credenciales RFID. De esta manera, el sistema se encarga de forma automática de toda la comunicación entre el lector y las tarjetas, sin necesidad de intervención externa en los procesos internos.

El chip MFRC522, desarrollado por [NXP Semiconductors](#), opera principalmente mediante una arquitectura basada en registros internos y comandos, los cuales permiten controlar cada una de sus funciones. A través de estos registros y comandos se ejecutan acciones como la detección de tarjetas, la autenticación, el envío y recepción de datos, y la gestión del sistema RFID en general.

Registro	Función	Uso en la práctica
CommandReg	Controla la ejecución de comandos del módulo	Iniciar lectura, escritura y reset del sistema
FIFODataReg	Buffer de datos de entrada y salida	Almacena datos enviados y recibidos con la tarjeta
Status1Reg	Estado general de la comunicación	Detecta si hay tarjeta presente o comunicación activa
Status2Reg	Estado del transmisor y autenticación	Verifica si la tarjeta está autenticada correctamente
ComIrqReg	Manejo de interrupciones	Detecta eventos como llegada de tarjeta o fin de operación
ErrorReg	Registro de errores	Indica fallos en comunicación o lectura
BitFramingReg	Control de estructura de bits	Ajusta sincronización en transmisión RFID
TxControlReg	Control de la antena RFID	Activa o desactiva el campo electromagnético
FIFOLevelReg	Nivel de datos en FIFO	Indica cuántos datos hay en buffer
CRCResultReg	Resultado de verificación CRC	Asegura integridad de datos transmitidos

Algunos de los comandos utilizados en la librería son:

Comando	Código	Función	Uso en la práctica
Idle	0000	No realiza ninguna acción	Cancela cualquier operación en curso
Mem	0001	Guarda datos en el buffer interno	Almacenamiento temporal de información
Generate RandomID	0010	Genera un ID aleatorio	No se usa directamente en esta práctica
CalcCRC	0011	Activa el cálculo CRC	Verificación de integridad de datos
Transmit	0100	Envía datos desde el FIFO	Envío de datos hacia la tarjeta RFID
NoCmdChange	0111	Mantiene el comando actual	Modificación de bits sin cambiar operación
Receive	1000	Activa el receptor	Espera respuesta de la tarjeta RFID
Transceive	1100	Envía y recibe datos automáticamente	Lectura y comunicación principal con tarjetas
MFAuthent	1110	Autenticación MIFARE	Acceso a sectores protegidos de la tarjeta

2.3 RFID Tag, Key Fob

Las credenciales RFID se pueden clasificar en dos tipos principales: **activas y pasivas**, dependiendo de la forma en la que obtienen su energía para funcionar.

Las credenciales **activas** cuentan internamente con una fuente de energía propia, generalmente una batería no reemplazable. Esto les permite transmitir datos de forma independiente y ofrecer mayor seguridad, además de tener un alcance más amplio en algunos casos. Sin embargo, su tiempo de vida es limitado debido al consumo de su batería. En el contexto de RFID básico, este tipo de credenciales no es el más común.

Por otro lado, las credenciales **pasivas** son las más utilizadas a nivel mundial debido a su bajo costo, alta durabilidad y tamaño reducido. Este tipo de tarjetas no cuenta con una fuente de energía interna, por lo que dependen completamente del campo electromagnético generado por el lector RFID para activarse. Cuando la tarjeta entra en el rango del lector, recibe energía suficiente para encenderse y transmitir su información de regreso al módulo.

Estas credenciales funcionan bajo la tecnología **MIFARE**, la cual está basada en el estándar **ISO/IEC 14443**, permitiendo la comunicación por radiofrecuencia entre el lector y la tarjeta de forma bidireccional mientras se encuentra dentro del campo electromagnético.

Entre las principales características de las credenciales RFID se encuentran:

Característica	Descripción	Característica
Memoria interna	Permite almacenar información como identificadores o datos de usuario	Memoria interna
Velocidad de transmisión de datos	Determina qué tan rápida es la comunicación entre la tarjeta y el lector RFID	Velocidad de transmisión de datos
Consumo energético	En tarjetas pasivas, depende completamente del campo generado por el lector RFID	Consumo energético

Estas características dependen del modelo seleccionado donde el principal proveedor a nivel mundial es NXP Semiconductors.

En este caso el modelo por defecto que se encuentra en los kits developer más comunes son [MF1S50YYX V1](#). Como se mencionó anteriormente, las credenciales RFID funcionan bajo el estándar **ISO/IEC 14443**, el cual define una serie de comandos que permiten la comunicación entre el lector y la tarjeta. Estos comandos son utilizados para realizar diferentes operaciones como la detección, selección, autenticación y lectura/escritura de datos dentro de la memoria de la credencial.

Cada comando tiene un código hexadecimal específico que el sistema interpreta para ejecutar una acción determinada dentro del proceso de comunicación RFID.

Entre los comandos más importantes utilizados en este tipo de sistemas se encuentran los siguientes:

Comando	Código hexadecimal	Función
Request (REQA)	26h (7 bits)	Detecta la presencia de una tarjeta en el campo del lector
Wake-Up (WUPA)	52h (7 bits)	Reactiva tarjetas en estado de reposo
Anticollision CL1	93h 20h	Detecta múltiples tarjetas y obtiene su UID
Select CL1	93h 70h	Selecciona una tarjeta específica en la primera capa

Anticollision CL2	95h 20h	Segunda etapa de anticollision (tarjetas avanzadas)
Select CL2	95h 70h	Selección de tarjeta en segundo nivel
Halt	50h 00h	Coloca la tarjeta en estado de bajo consumo
Authentication Key A	60h	Autenticación con clave A
Authentication Key B	61h	Autenticación con clave B
Read	30h	Lectura de bloques de memoria
Write	A0h	Escritura de datos en la tarjeta
Increment	C1h	Incrementa valores almacenados
Decrement	C0h	Disminuye valores almacenados
Restore	C2h	Restaura valores desde backup
Transfer	B0h	Transfiere datos al bloque de memoria

Para una descripción mas detallada es necesario ver [MIFARE Classic commands](#) Capitulo 12

3. Arquitectura del Software

La implementación del sistema RFID se basa en el uso de una librería previamente desarrollada por [Tasm-Devil](#), debido a que actualmente no existe una versión oficial completamente optimizada de MicroPython para el módulo **RC522 ESP32**.

Por esta razón, se utiliza esta librería como base funcional, la cual permite gestionar la comunicación entre el microcontrolador y el módulo RFID mediante SPI, además de proporcionar las funciones necesarias para la lectura, escritura y autenticación de credenciales. esto debido a fines prácticos.

Archivo	Responsabilidad
RFID.py	Lógica principal del sistema: detección de tarjetas, autenticación, lectura y escritura de datos RFID
mfrc522.py	Librería de bajo nivel encargada de la configuración del módulo, comunicación SPI y control de registros internos del MFRC522

4. Descripción Técnica Detallada

4.1 Inicialización del módulo RFID (MFRC522)

El sistema RFID inicia mediante la creación de una instancia del módulo **RC522**, el cual se encarga de toda la comunicación con las credenciales.

Durante esta etapa se configuran los pines de comunicación SPI (SCK, MOSI, MISO, RST y CS), los cuales permiten la conexión directa entre el ESP32-C6 y el lector RFID.

El objetivo de esta fase es preparar el sistema para activar la antena interna del lector y habilitar la detección de tarjetas dentro del rango de lectura.

4.2 Configuración del sistema SPI

El módulo RC522 utiliza el protocolo SPI para comunicarse con el microcontrolador.

El ESP32-C6 actúa como maestro del bus SPI, mientras que el lector RFID funciona como esclavo.

A través de esta comunicación se envían comandos, datos y configuraciones hacia los registros internos del módulo.

4.3 Lógica principal del sistema — RFID.py

Este es el archivo principal del sistema RFID, donde se ejecuta el ciclo completo de funcionamiento.

El programa permanece en un bucle infinito que permite la detección constante de tarjetas RFID dentro del rango del lector.

```
from mfrc522 import MFRC522
from machine import Pin
import time

rdr = MFRC522(sck=18, mosi=23, miso=19, rst=22, cs=5)

print("Tarjeta RFID")

while True:
    (stat, tag_type) = rdr.request(rdr.REQIDL)

    if stat == rdr.OK:
        print("Tarjeta detectada")

        (stat, uid) = rdr.anticoll()

        if stat == rdr.OK:
            print("UID:", uid)

            if rdr.select_tag(uid) == rdr.OK:

                key = [255, 255, 255, 255, 255, 255]

                if rdr.auth(rdr.AUTHENT1A, 8, key, uid) == rdr.OK:

                    print("Autenticado correctamente")

                    data = "Jose Angel"
                    data_bytes = [ord(c) for c in data]

                    rdr.write(8, data_bytes)
```

```

        print("Datos escritos")

        read_data = rdr.read(8)

        if read_data:
            texto = "".join([chr(i) for i in read_data])
            print("Leido:", texto)

        rdr.stop_cryptol()

    time.sleep(0.5)

```

4.4 Explicación del flujo del código

4.4.1 Detección de tarjeta (request)

```
(stat, tag_type) = rdr.request(rdr.REQIDL)
```

La función `request()` envía continuamente una petición de búsqueda al lector RC522 para detectar la presencia de tarjetas RFID dentro del campo electromagnético generado por la antena.

El parámetro `REQIDL` indica que se buscarán tarjetas en estado de reposo (Idle State).

La función retorna:

- **stat**: Estado de la operación (verifica si ocurrió un error de comunicación CRC)
- **tag_type**: Tipo de tarjeta detectada.

Si el valor de **stat** coincide con `rdr.OK`, significa que una tarjeta RFID ha sido encontrada correctamente.

```

if stat == rdr.OK:
    print("Tarjeta detectada")

```

4.4.2 Obtención del UID (anticoll)

Una vez detectada la tarjeta, se ejecuta el procedimiento de anticolisión.

```
(stat, uid) = rdr.anticoll()
```

El proceso de anticolisión permite identificar una tarjeta específica cuando existen varias tarjetas dentro del alcance del lector.

La función retorna:

- **stat**: Estado de la operación (verifica si ocurrió un error de comunicación CRC)
- **uid**: Identificador único de la tarjeta.

Posteriormente el UID es mostrado en consola.

```
print("UID:", uid)
```

El UID es un número único programado por el fabricante y permite identificar individualmente cada tarjeta RFID.

4.4.3 Selección de tarjeta

```
rdr.select_tag(uid)
```

Esta función nos permite detectar la credencial con la que queramos trabajar debido a que es posible a que se empalmen varias tarjetas al mismo tiempo y para recibir o enviar la

información correcta es necesario identificar primero la tarjeta específica que queramos identificar.

4.4.4 Autenticación

```
rdr.auth(rdr.AUTHENT1A, 8, key, uid)
```

Antes de acceder a la memoria de la tarjeta, es necesario realizar una autenticación. Aquí se utiliza una clave de seguridad para verificar si el lector tiene permiso de leer o escribir en la tarjeta. Si la autenticación es correcta, se habilita el acceso a los bloques de memoria.

Es importante recalcar que la tarjeta RFID está organizada internamente en sectores y bloques de memoria. Cada tarjeta cuenta con **16 sectores**, y cada sector está compuesto por **4 bloques de 16 bytes** cada uno. Sin embargo, no todos los bloques pueden ser utilizados libremente. El primer bloque de cada sector está reservado para almacenar información de control como claves de autenticación y permisos de acceso, por lo que no se utiliza para guardar datos. Además, **el sector 0 está completamente protegido, ya que contiene** información crítica de la tarjeta, como el UID del fabricante, datos del modelo y configuraciones de seguridad, por lo que no puede ser modificado ni utilizado para almacenamiento de datos.

Sector	Block	Byte Number within a Block																Description
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
15	3	Key A				Access Bits				Key B								Sector Trailer 15
	2																	Data
	1																	Data
	0																	Data
14	3	Key A				Access Bits				Key B								Sector Trailer 14
	2																	Data
	1																	Data
	0																	Data
:	:																	
:	:																	
:	:																	
1	3	Key A				Access Bits				Key B								Sector Trailer 1
	2																	Data
	1																	Data
	0																	Data
0	3	Key A				Access Bits				Key B								Sector Trailer 0
	2																	Data
	1																	Data
	0	Manufacturer Data																Manufacturer Block

Es debido a ello que utiliza el sector 8 para guardar la información del sector 0 y mandarla nuestro modulo RFID, este sector es opción teniendo la posibilidad de utilizar otro.

4.2.4 Clave de Autenticación

```
key = [255, 255, 255, 255, 255, 255]
```

Las tarjetas MIFARE Classic utilizan claves de seguridad de 6 bytes para permitir el acceso a sus sectores de memoria.

La clave:

```
FF FF FF FF FF FF
```

corresponde a la clave predeterminada de fábrica utilizada en muchas tarjetas RFID nuevas.

4.2.5 Autenticación del Bloque

```
if rdr.auth(rdr.AUTHENT1A, 8, key, uid) == rdr.OK:
```

Antes de leer o escribir información es obligatorio autenticar el bloque de memoria correspondiente.

Parámetros utilizados:

- **AUTHENT1A**: Autenticación mediante Key A.
- **8**: Bloque de memoria a utilizar (este bloque es opcional).
- **key**: Clave de acceso.
- **uid**: Identificador de la tarjeta.

Si la autenticación es exitosa se habilita el acceso al bloque seleccionado.

```
print("Autenticado correctamente")
```

4.2.6 Escritura de Datos

```
data = "Jose Angel"
data_bytes = [ord(c) for c in data]
```

Primero se define el texto que será almacenado en la tarjeta RFID.

Posteriormente cada carácter es convertido a su representación ASCII mediante la función `ord()`, generando una lista de bytes compatible con el formato de almacenamiento de la memoria MIFARE.

La escritura se realiza mediante:

```
rdr.write(8, data_bytes)
```

Donde:

- **8** corresponde al bloque de memoria.
- **data_bytes** contiene la información a almacenar.

Finalmente se confirma la operación:

```
print("Datos escritos")
```

4.2.7 Lectura de Datos

Después de escribir la información se realiza una lectura de verificación, para ello es necesario seleccionar el bloque donde obtendremos nuestra información en este caso el número 8.

```
if read_data:
```

```

        texto = "".join([chr(i) for i in read_data])
        print("Leido:", texto)

        rdr.stop_cryptol()

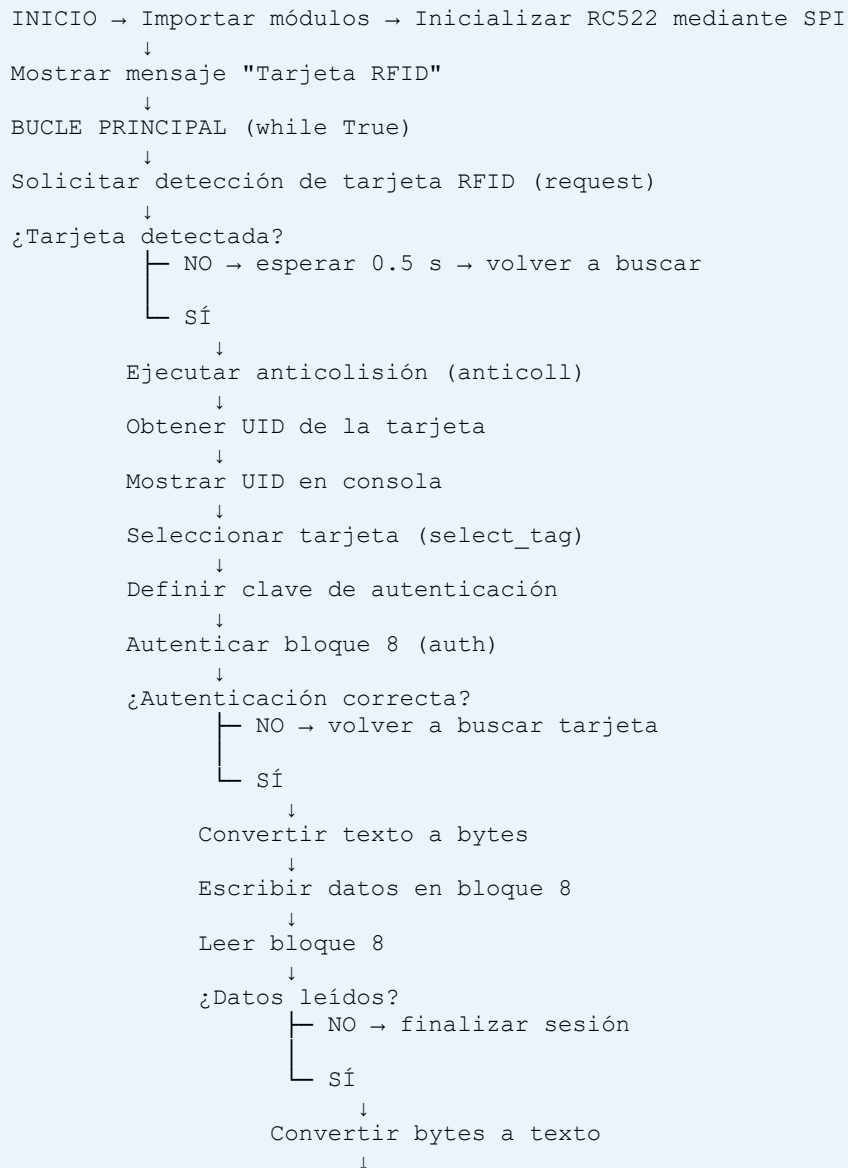
time.sleep(0.5)

```

Despues damos un pequeño intervalo de medio segundo para evitar que nuestro modulo siga leyendo información de nuestra credencial.

5. Flujo del sistema.

Acontinuacion se muestra el flujo de nuestro sistema.



```
Mostrar información leída
↓
Finalizar autenticación
(stop_cryptol)
↓
Esperar 0.5 s
↓
Volver a buscar tarjeta
```

6. RFID — Conceptos Clave

La tecnología RFID (Radio Frequency Identification) es un método de identificación automática que utiliza ondas de radio para intercambiar información entre un lector y una credencial electrónica sin necesidad de contacto físico. En esta práctica se utiliza el módulo MFRC522, el cual opera a una frecuencia de 13.56 MHz y es compatible con tarjetas y llaveros RFID basados en la tecnología **MIFARE** bajo el estándar **ISO/IEC 14443 Tipo A**.

El módulo RC522 incorpora una antena y un circuito integrado encargado de generar el campo electromagnético necesario para detectar y energizar credenciales RFID pasivas. Una vez energizada la tarjeta, se establece una comunicación bidireccional que permite identificar la credencial y acceder a la información almacenada en su memoria interna.

En esta práctica:

- El ESP32-C6 se comunica con el módulo MFRC522 mediante el protocolo SPI.
- El RC522 genera el campo electromagnético necesario para detectar tarjetas RFID cercanas.
- Cada credencial RFID posee un identificador único (UID) utilizado para distinguirla de otras tarjetas.
- El sistema realiza procesos de detección, selección y autenticación antes de acceder a la memoria de la tarjeta.
- La lectura y escritura de información se realiza mediante comandos compatibles con la tecnología MIFARE.

Cada comunicación RFID sigue una lógica simplificada:

[Detección de tarjeta] → [Lectura de UID] → [Autenticación] → [Lectura/Escritura] → [Fin de comunicación]

Por otra parte, el módulo **RC522** funciona mediante una arquitectura basada en registros y comandos internos. Estos registros permiten controlar aspectos como la transmisión de datos, recepción de información, manejo de interrupciones, control de la antena y verificación de errores durante la comunicación. Algunas de las funciones de nuestra librería son:

- **request()** detecta la presencia de una credencial RFID dentro del rango de lectura.
- **anticoll()** obtiene el UID de la tarjeta y evita conflictos entre múltiples credenciales.
- **select_tag()** selecciona una tarjeta específica para trabajar con ella.
- **auth()** realiza la autenticación utilizando una clave de acceso.

- **write()** almacena información dentro de un bloque de memoria RFID.
- **read()** recupera la información almacenada previamente.
- **stop_crypto1()** finaliza la sesión de autenticación de forma segura.

Gracias a esta arquitectura, el ESP32-C6 puede identificar credenciales RFID, validar permisos de acceso y almacenar o recuperar información desde la memoria interna de la tarjeta de manera rápida y confiable.

Además, es importante considerar la estructura interna de las tarjetas MIFARE Classic 1K utilizadas en esta práctica. Estas credenciales disponen de **16 sectores**, donde cada sector contiene **4 bloques de memoria de 16 bytes cada uno**. Sin embargo, algunos bloques están reservados para información de control y seguridad, por lo que no todos pueden utilizarse para almacenar datos del usuario. El **sector 0** contiene información del fabricante y el UID de la tarjeta, mientras que el último bloque de cada sector se utiliza para almacenar claves de autenticación y permisos de acceso.

Gracias a esta organización de memoria, las credenciales RFID permiten implementar sistemas de identificación, control de acceso y almacenamiento de información de forma segura y eficiente.

7. Dependencias y Bibliotecas

Módulo/Biblioteca	Origen	Uso en el proyecto
machine	MicroPython stdlib	Permite el control del hardware del ESP32, incluyendo pines GPIO y comunicación SPI con el módulo RFID
Pin	MicroPython stdlib	Configuración de pines digitales para la conexión con el módulo RC522
SPI	MicroPython stdlib	Protocolo de comunicación serial utilizado entre el ESP32 y el lector RFID MFRC522
os (uname)	MicroPython stdlib	Identificación de la plataforma para configurar automáticamente el tipo de comunicación SPI
time	MicroPython stdlib	Control de retardos en el ciclo continuo de lectura de tarjetas RFID
mfr522	Librería externa	Librería principal que implementa el control del módulo RC522 para lectura, escritura y autenticación RFID

8. Consideraciones importantes.

Para el correcto funcionamiento del sistema RFID con el módulo MFRC522 se deben tomar en cuenta las siguientes consideraciones:

- Utilizar **Python 3.5 o superior** en el entorno de desarrollo.
- Tener instaladas las librerías necesarias en la computadora para pruebas y desarrollo.
- Guardar la librería **mfrc522.py directamente en el ESP32**, asegurando que pueda ser importada correctamente.
- Verificar el **correcto cableado del módulo RFID**, respetando los pines definidos en el código (SCK, MOSI, MISO, RST y CS).
- Alimentar el módulo RFID con **3.3V estable** para evitar fallos de lectura o escritura.
- Utilizar tarjetas compatibles con el estándar **MIFARE 13.56 MHz**.
- Realizar pruebas iniciales de lectura de UID antes de implementar escritura o autenticación.

9. Conclusiones

Esta práctica demuestra la implementación exitosa de un sistema de lectura, autenticación, escritura y verificación de datos RFID utilizando el módulo RC522 junto con el ESP32 en MicroPython.

Principales resultados:

- El módulo MFRC522 permite la comunicación eficiente con tarjetas RFID mediante el protocolo SPI.
- La librería personalizada facilita el control de registros internos para realizar lectura, escritura y autenticación.
- El proceso de detección mediante request() y anticoll() permite identificar credenciales de forma confiable.
- La autenticación por sectores garantiza el acceso seguro a la memoria de las tarjetas RFID.
- La escritura y lectura de datos demuestra la capacidad de almacenamiento y recuperación de información en tiempo real.
- El uso de tarjetas MIFARE permite una estructura de memoria organizada en sectores y bloques.
- El sistema RFID representa una solución práctica para aplicaciones de control de acceso e identificación sin contacto.

10. Bibliografía

1. NXP Semiconductors. (s. f.). *MF1S50YYX_V1 MIFARE Classic EV1 1K – Data sheet*. https://www.nxp.com/docs/en/data-sheet/MF1S50YYX_V1.pdf
2. NXP Semiconductors. (s. f.). *AN10833: MIFARE Classic protocol overview*. <https://www.nxp.com/docs/en/application-note/AN10833.pdf>
3. NXP Semiconductors. (s. f.). *RC522 RFID reader module data sheet*. <https://www.alldatasheet.es/datasheet-pdf/view/346109/NXP/RC522.html>
4. Tasm-Devil. (s. f.). *micropython-mfrc522-esp32*. GitHub. <https://github.com/Tasm-Devil/micropython-mfrc522-esp32>

11. Anexo

Librería mfrc522.py utilizada. Revisar en conjunto con los temas 2 y 4, acompañada de la bibliografía 1 y 2, para una mejor comprensión.

```
from machine import Pin, SPI
from os import uname

class MFRC522:

    OK = 0
    NOTAGERR = 1
    ERR = 2

    REQIDL = 0x26
    REQALL = 0x52
    AUTHENT1A = 0x60
    AUTHENT1B = 0x61

    def __init__(self, sck, mosi, miso, rst, cs):

        self.sck = Pin(sck, Pin.OUT)
        self.mosi = Pin(mosi, Pin.OUT)
        self.miso = Pin(miso)
        self.rst = Pin(rst, Pin.OUT)
        self.cs = Pin(cs, Pin.OUT)

        self.rst.value(0)
        self.cs.value(1)

        board = uname()[0]

        if board == 'WiPy' or board == 'LoPy' or board == 'FiPy':
            self.spi = SPI(0)
            self.spi.init(SPI.MASTER, baudrate=1000000, pins=(self.sck,
self.mosi, self.miso))
        elif board == 'esp8266' or board == 'esp32':
            self.spi = SPI(baudrate=100000, polarity=0, phase=0,
sck=self.sck, mosi=self.mosi, miso=self.miso)
            self.spi.init()
        else:
            raise RuntimeError("Unsupported platform")

        self.rst.value(1)
        self.init()

    def _wreg(self, reg, val):

        self.cs.value(0)
        self.spi.write(b'%c' % int(0xff & ((reg << 1) & 0x7e)))
        self.spi.write(b'%c' % int(0xff & val))
        self.cs.value(1)

    def _rreg(self, reg):

        self.cs.value(0)
        self.spi.write(b'%c' % int(0xff & (((reg << 1) & 0x7e) | 0x80)))
        val = self.spi.read(1)
        self.cs.value(1)

        return val[0]
```

```

def _sflags(self, reg, mask):
    self._wreg(reg, self._rreg(reg) | mask)

def _cflags(self, reg, mask):
    self._wreg(reg, self._rreg(reg) & (~mask))

def _tocard(self, cmd, send):

    recv = []
    bits = irq_en = wait_irq = n = 0
    stat = self.ERR

    if cmd == 0x0E:
        irq_en = 0x12
        wait_irq = 0x10
    elif cmd == 0x0C:
        irq_en = 0x77
        wait_irq = 0x30

    self._wreg(0x02, irq_en | 0x80)
    self._cflags(0x04, 0x80)
    self._sflags(0x0A, 0x80)
    self._wreg(0x01, 0x00)

    for c in send:
        self._wreg(0x09, c)
    self._wreg(0x01, cmd)

    if cmd == 0x0C:
        self._sflags(0x0D, 0x80)

    i = 2000
    while True:
        n = self._rreg(0x04)
        i -= 1
        if ~(i != 0) and ~(n & 0x01) and ~(n & wait_irq):
            break

    self._cflags(0x0D, 0x80)

    if i:
        if (self._rreg(0x06) & 0x1B) == 0x00:
            stat = self.OK

            if n & irq_en & 0x01:
                stat = self.NOTAGERR
            elif cmd == 0x0C:
                n = self._rreg(0x0A)
                lbits = self._rreg(0x0C) & 0x07
                if lbits != 0:
                    bits = (n - 1) * 8 + lbits
                else:
                    bits = n * 8

                if n == 0:
                    n = 1
                elif n > 16:
                    n = 16

                for _ in range(n):
                    recv.append(self._rreg(0x09))
            else:

```

```

        stat = self.ERR

        return stat, recv, bits

def _crc(self, data):

    self._cflags(0x05, 0x04)
    self._sflags(0x0A, 0x80)

    for c in data:
        self._wreg(0x09, c)

    self._wreg(0x01, 0x03)

    i = 0xFF
    while True:
        n = self._rreg(0x05)
        i -= 1
        if not ((i != 0) and not (n & 0x04)):
            break

    return [self._rreg(0x22), self._rreg(0x21)]

def init(self):

    self.reset()
    self._wreg(0x2A, 0x8D)
    self._wreg(0x2B, 0x3E)
    self._wreg(0x2D, 30)
    self._wreg(0x2C, 0)
    self._wreg(0x15, 0x40)
    self._wreg(0x11, 0x3D)
    self.antenna_on()

def reset(self):
    self._wreg(0x01, 0x0F)

def antenna_on(self, on=True):

    if on and ~(self._rreg(0x14) & 0x03):
        self._sflags(0x14, 0x03)
    else:
        self._cflags(0x14, 0x03)

def request(self, mode):

    self._wreg(0x0D, 0x07)
    (stat, recv, bits) = self._tocard(0x0C, [mode])

    if (stat != self.OK) | (bits != 0x10):
        stat = self.ERR

    return stat, bits

def anticoll(self):

    ser_chk = 0
    ser = [0x93, 0x20]

    self._wreg(0x0D, 0x00)
    (stat, recv, bits) = self._tocard(0x0C, ser)

    if stat == self.OK:

```

```

        if len(recv) == 5:
            for i in range(4):
                ser_chk = ser_chk ^ recv[i]
            if ser_chk != recv[4]:
                stat = self.ERR
        else:
            stat = self.ERR

    return stat, recv

def select_tag(self, ser):

    buf = [0x93, 0x70] + ser[:5]
    buf += self._crc(buf)
    (stat, recv, bits) = self._tocard(0x0C, buf)
    return self.OK if (stat == self.OK) and (bits == 0x18) else
self.ERR

def auth(self, mode, addr, sect, ser):
    return self._tocard(0x0E, [mode, addr] + sect + ser[:4])[0]

def stop_crypto1(self):
    self._cflags(0x08, 0x08)

def read(self, addr):

    data = [0x30, addr]
    data += self._crc(data)
    (stat, recv, _) = self._tocard(0x0C, data)
    return recv if stat == self.OK else None

def write(self, addr, data):

    buf = [0xA0, addr]
    buf += self._crc(buf)
    (stat, recv, bits) = self._tocard(0x0C, buf)

    if not (stat == self.OK) or not (bits == 4) or not ((recv[0] &
0x0F) == 0x0A):
        stat = self.ERR
    else:
        buf = []
        for i in range(16):
            buf.append(data[i])
        buf += self._crc(buf)
        (stat, recv, bits) = self._tocard(0x0C, buf)
        if not (stat == self.OK) or not (bits == 4) or not ((recv[0]
& 0x0F) == 0x0A):
            stat = self.ERR

    return stat

```